

Setting Up GitHub for Your Walk Journal

A complete, step-by-step guide — from creating an account to hosting the app online and syncing your walks between devices.

No coding experience required. Set aside about 20 minutes.

Works for any Walk Journal route (Speyside Way, South West Coast Path, and others).

0 What you are setting up, and why

GitHub is a free service that does two separate jobs for this app:

- **Hosts the app online** so you can open it from any device by visiting a web address, and install it like a normal app (this uses a feature called *GitHub Pages*).
- **Stores your walks in the cloud** so they back up and sync between your phone, tablet and computer (this uses a *repository* plus a secure *access token*).

To do both safely you will create **two repositories** (a repository is just a folder of files that GitHub keeps for you):

- A **public** repository that holds the app's files — this is required for free online hosting.
- A **private** repository that holds **your walk data** — this keeps your journal entries and photos visible to you alone.

Why two repositories? (Please read)

Anything in a **public** repository can be read by anyone on the internet. Free online hosting only works with a public repository, so the app's *code* goes there — but it contains none of your personal data.

Your walks, notes and photos are written only to the **private** repository, which no one else can see. The app reads and writes that private data through your personal access token, never through the public web address.

What you will need:

- A computer with a web browser and an email address.
- The app's built files — the contents of the **dist** folder (Part 3 explains how to get them).

1 Create a GitHub account

- 1 In your web browser, go to github.com/signup.
- 2 Enter your email address, then choose a password and a **username**. Your username becomes part of your app's web address (for example `https://YOUR-USERNAME.github.io/...`), so pick something you are happy to share publicly.
- 3 Complete the puzzle to prove you are human, then enter the code GitHub emails to you.
- 4 When asked which plan you want, choose the **Free** plan — it is all you need.
- 5 Open the verification email from GitHub and click the link to confirm your address.

Tip

Write down your username and store your password in a password manager. You will need the username again in Parts 4 and 6.

2 Create the two repositories

2a — The public repository (for the app)

- 1 Sign in to GitHub. At the top-right, click the **+** icon and choose **New repository**.

- 2 **Repository name:** type a short, lowercase name such as `walk-journal-app` (no spaces).
- 3 **Visibility:** choose **Public**. (This is required for free online hosting.)
- 4 Tick **Add a README file** — this gives the repository a starting point.
- 5 Click **Create repository**.

2b — The private repository (for your walks)

- 1 Click the **+** icon again and choose **New repository**.
- 2 **Repository name:** type a name for your journal, such as `walk-journal-data`. Remember this name — you will type it into the app in Part 6.
- 3 **Visibility:** choose **Private**. (This keeps your walks and photos visible to you only.)
- 4 Tick **Add a README file**.
- 5 Click **Create repository**.

You do not need to make any folders

Leave the private repository empty apart from the README. The first time the app syncs, it automatically creates the folders it needs (`walks/`, `photos/` and `gpx/`).

3 Add the app files to the public repository

First, get the built files. The app is packaged into a folder called `dist`. If you already have it, skip to *Upload*. To build it yourself, open a terminal in the app's project folder and run:

```
npm install (first time only)
npm run build
```

After building, the `dist` folder contains these items:

```
index.html
manifest.json
sw.js
icons/ (favicon-32.png, icon-192.png, icon-512.png,
icon-maskable-192.png, icon-maskable-512.png)
```

Upload them through the website:

- 1 Open your **public** app repository on GitHub.
- 2 Click **Add file** > **Upload files**.
- 3 On your computer, open the `dist` folder. Select the files *inside* it (`index.html`, `manifest.json`, `sw.js`) *and* the `icons` folder, then drag them onto the upload area. GitHub keeps the `icons` folder structure for you.
- 4 Scroll down, type a short message such as “Add app”, then click **Commit changes**.

Important

Upload the **contents** of `dist`, not the `dist` folder itself. `index.html` must end up at the **top level** of the repository, or the site will not load.

Updating later

Whenever you rebuild a new version of the app, repeat this upload. Uploading a file with the same name simply replaces the old one.

4 Make the app available online (GitHub Pages)

- 1 In your **public** app repository, click **Settings** (top menu).
- 2 In the left sidebar, click **Pages**.
- 3 Under **Build and deployment** > **Source**, choose **Deploy from a branch**.
- 4 Set **Branch** to **main** and the folder to **/ (root)**, then click **Save**.
- 5 Wait one or two minutes, then refresh the page. GitHub shows a banner: “Your site is live at” followed by an address like `https://YOUR-USERNAME.github.io/walk-journal-app/`.
- 6 Open that address in your browser — the app loads.

Install it like an app:

- **On a phone:** open the address, then use the browser's share menu and choose **Add to Home Screen**.
- **On a desktop** (Chrome or Edge): click the install icon in the address bar, or the app's own **Install** button.

Offline use

The first time you open the app online, it quietly saves a copy of itself. After that it keeps working even with no signal — handy on the trail.

5 Generate an access token (PAT) for syncing walks

The app uploads your walks using a **Personal Access Token** (PAT) — a long secret string that grants write access to your **private** journal repository only. The app expects a **fine-grained** token with **Contents: Read and write** permission.

- 1 Go to github.com/settings/personal-access-tokens (or: your avatar > **Settings** > **Developer settings** > **Personal access tokens** > **Fine-grained tokens**).
- 2 Click **Generate new token**.
- 3 **Token name:** something memorable, e.g. `Walk Journal sync`.
- 4 **Expiration:** pick a date (for example 90 days, or 1 year). See the note below about renewing it.
- 5 **Resource owner:** your own account.
- 6 **Repository access:** choose **Only select repositories**, then select your **private** journal repository (e.g. `walk-journal-data`).
- 7 **Permissions:** expand **Repository permissions**, find **Contents**, and set it to **Read and write**. (**Metadata** will switch to *Read-only* by itself — leave it; it is required.)
- 8 Click **Generate token**.
- 9 **Copy the token now.** It begins with `github_pat_` and is shown only once. Paste it somewhere safe for a moment — you will enter it into the app in Part 6.

Tokens expire — that is normal

When the expiry date passes, syncing stops working until you create a new token. Repeat this part, then update the app using **Reset PAT** (see Part 6). Setting a long expiry means fewer renewals.

Keep it secret

Treat the token like a password. Do not share it, email it, or put it into any repository. If it ever leaks, delete it on GitHub (**Settings** > **Developer settings**) and generate a new one.

Alternative: a classic token

If you prefer, you can use a *classic* token instead: **Settings** > **Developer settings** > **Personal access tokens** > **Tokens (classic)** > **Generate new token**, tick the **repo** scope, generate, and copy it. Classic tokens grant access to *all* your repositories, so the fine-grained token above is the safer choice.

6 Connect the app to your journal repository

- 1 Open the app (online, or the installed version).
- 2 Open the **Sync & Backup** panel by clicking the sync / cloud status area.
- 3 Under **No cloud sync configured**, fill in the three boxes:
 - **GitHub Personal Access Token** — paste the `github_pat_...` token from Part 5.
 - **Repository Owner** — your GitHub username (exactly as it appears, capitals matter).
 - **Repository Name** — your **private** journal repository's name (e.g. `walk-journal-data`).
- 4 Click **Validate**. The app checks the token and repository and confirms with a message like “Connected to your-username/walk-journal-data”.
- 5 Click **Connect & pull**. The app saves the settings and performs a first sync.

That is it. From now on, every walk you log is queued and uploaded automatically whenever you are online. The app also keeps everything on your device, so it works offline and catches up later.

Renewing an expired token

When a token expires, open **Sync & Backup** > **Reset PAT**, paste a freshly generated token (Part 5), and you are reconnected — no need to set up anything else again.

Adding another device

To sync your phone and computer, open the app on each device and connect it (Part 6) to the **same private repository**. You can reuse the same token or generate one per device.

7 If something goes wrong

Symptom	What to check
---------	---------------

"Validation failed" when connecting	Check that the Owner is your exact username and the Repository Name matches exactly (capitals matter). Make sure the token has Contents: Read and write on that repository and has not expired.
The app's web address shows 404 / will not load	Wait a couple of minutes after enabling Pages. Confirm <code>index.html</code> is at the top level of the public repo (not inside a <code>dist</code> folder), and that Pages Source is set to main / root .
Icons missing, or offline mode not working	Make sure the <code>icons</code> folder, <code>sw.js</code> and <code>manifest.json</code> were all uploaded alongside <code>index.html</code> .
Walks not showing on another device	Open the app on that device and connect it to the same private repository (Part 6). Each device must be connected once.

8 Quick reference

- **Your app address:** `https://YOUR-USERNAME.github.io/PUBLIC-REPO/`
- **Public repo** (app files, GitHub Pages): `YOUR-USERNAME/PUBLIC-REPO`
- **Private repo** (your walks): `YOUR-USERNAME/PRIVATE-REPO`
- **Token:** fine-grained, **Contents: Read and write**, private repo only — entered in the app's **Sync & Backup** panel.

One-line summary

Public repo hosts the app online; private repo + token stores your walks privately. The app's code is public; your journal is not.